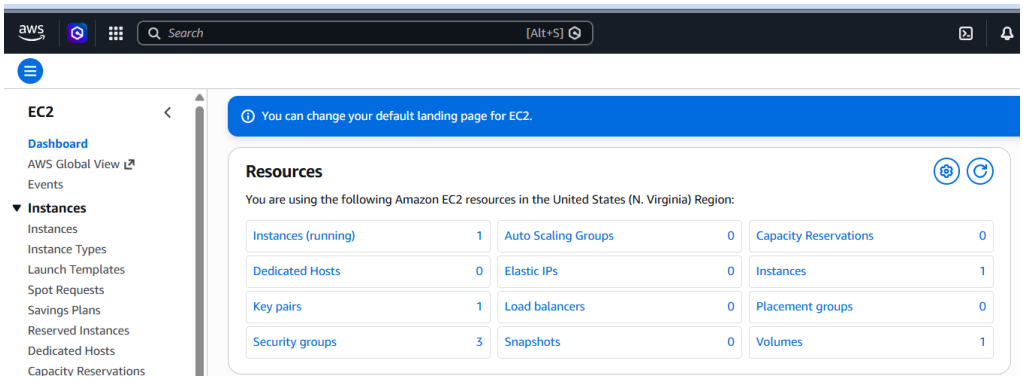
Assignment 7: Deploy a simple Web Application on EC2 server using AWS Database services

Phạm Trần Gia Hưng - 2331200153

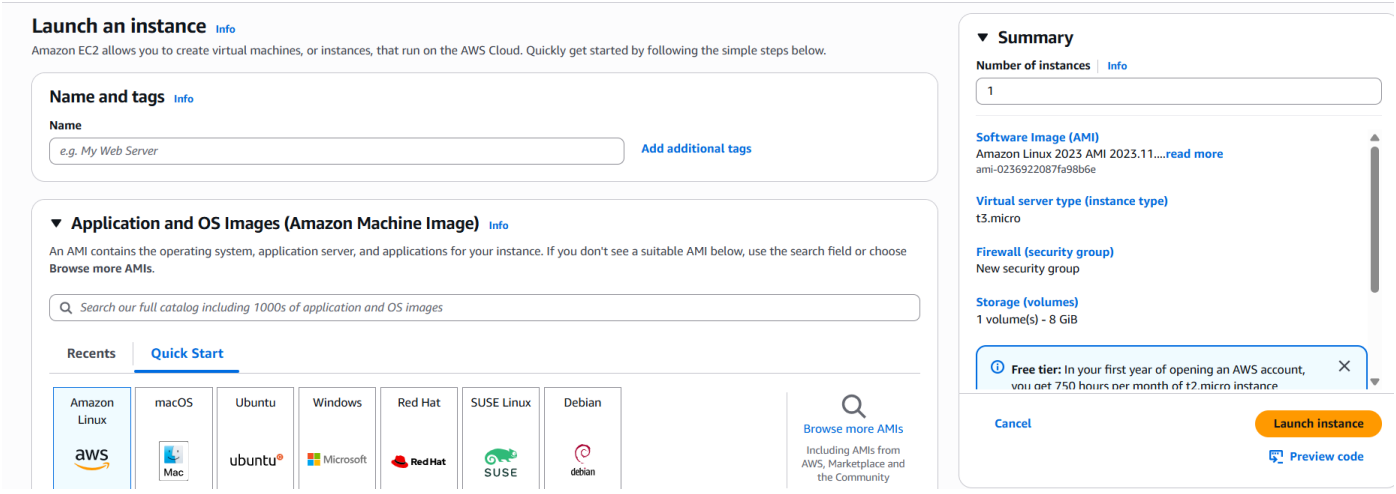
Task 1: Create an public EC2 Ubuntu user instance

● Instructions:

1. Navigate to the EC2 service.

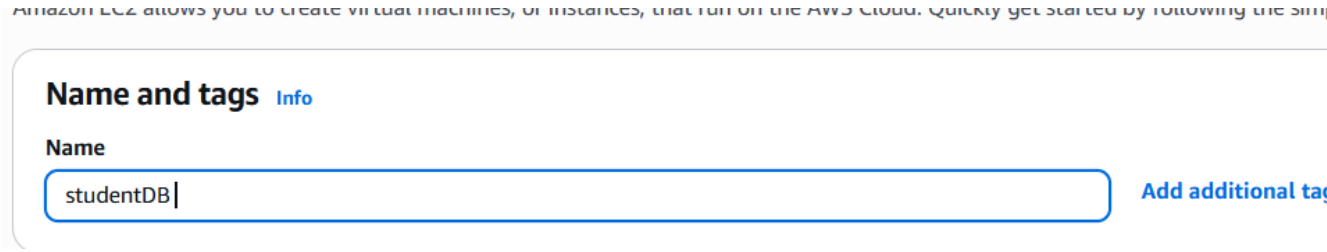


2. Click on Launch Instance

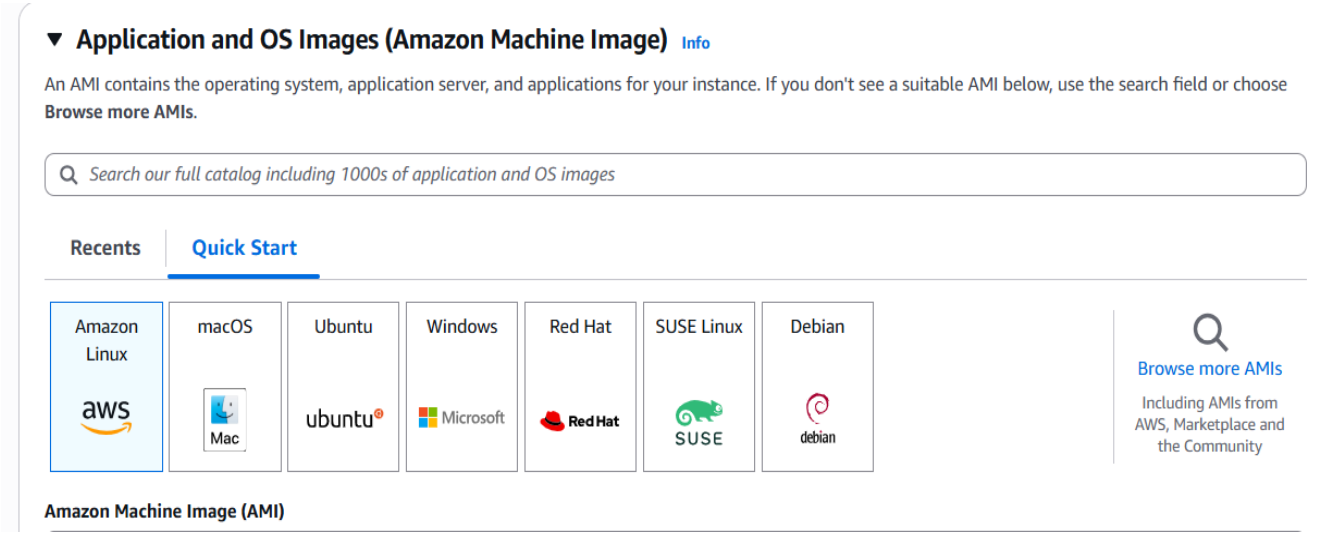


3. Launch EC2 (Public Instance):

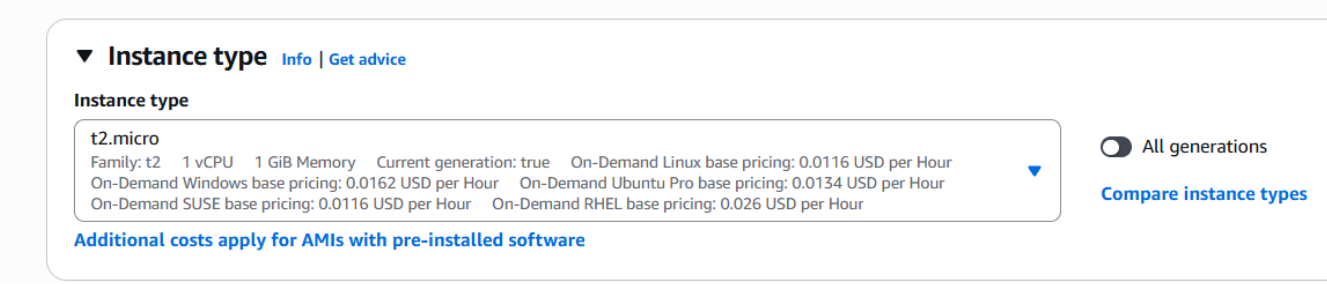
- ◆ Name: studentDB



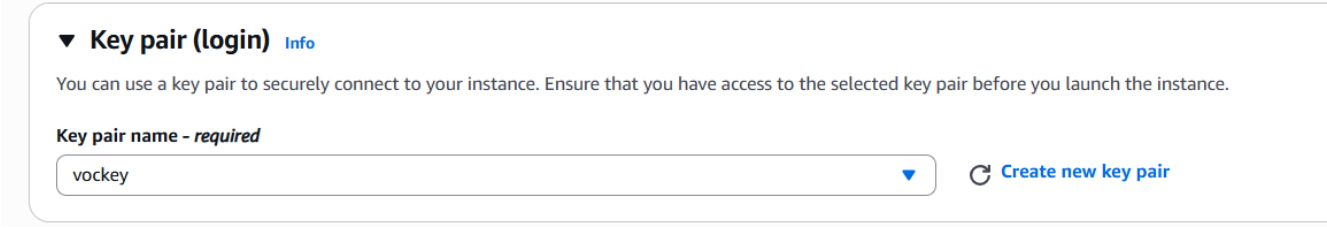
◆ Application and OS Images (AMI): Select Ubuntu (Free tier eligible).



◆ Instance type: Select t2.micro (Free tier eligible).



◆ Key pair (login): select an existing key pair.



◆ Network settings: Click Edit.

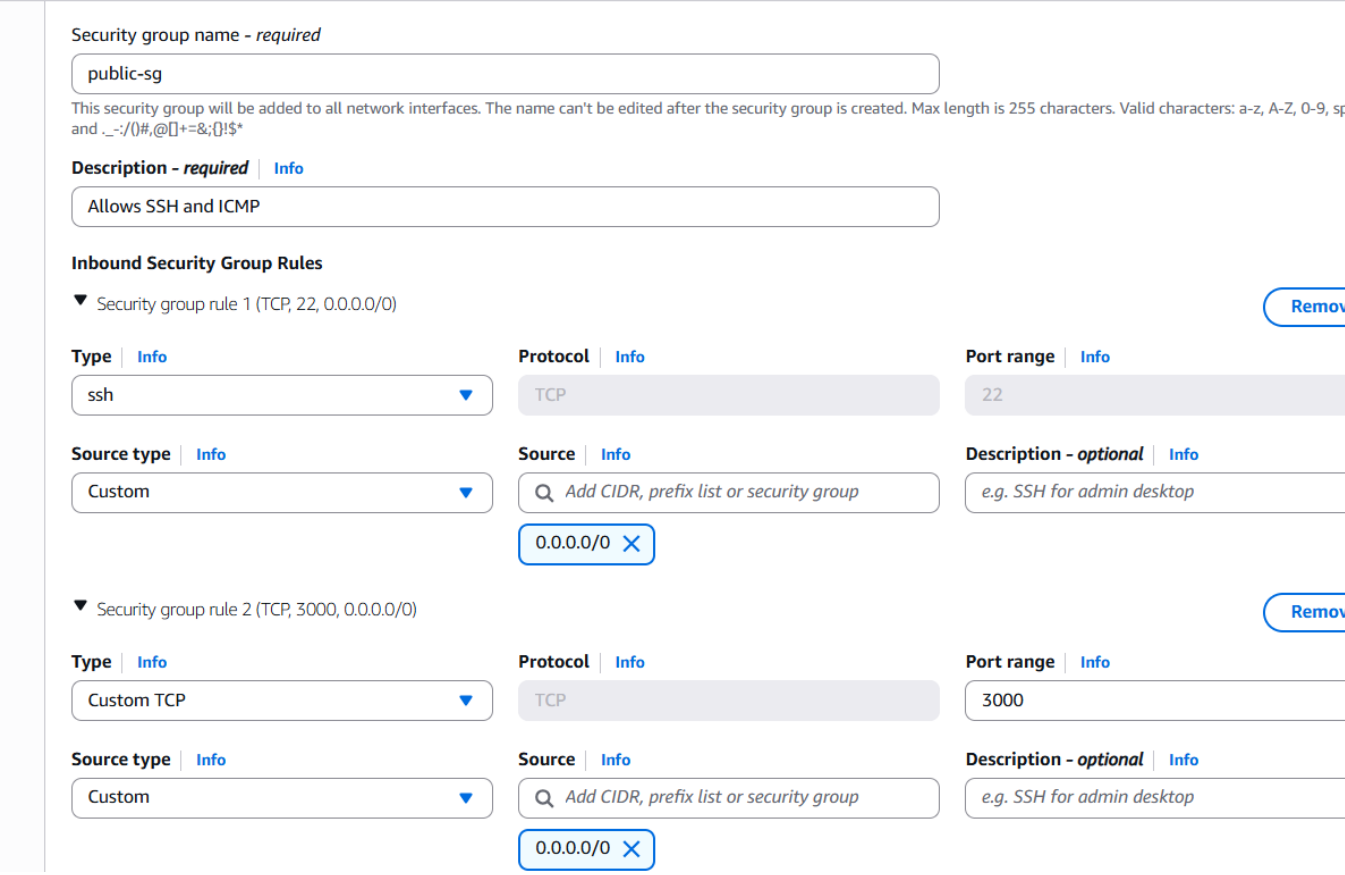
- VPC: Select default VPC.
- Subnet: Select any existing public-subnet of US-east-1a

- Auto-assign public IP: Select Enable.

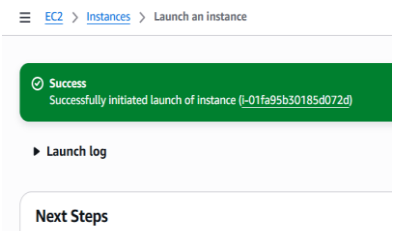


◆ Firewall (security groups):

- Select Create a new security group.
- Security group name: public-sg
- Description: Allows SSH and ICMP
- Inbound rules:
  - Rule 1: Type SSH, Source C**ustom** use 0.0.0.0/0. Port 22
  - Rule 2: Click Add security group rule. Type **CustomTCP**, Source Custom 0.0.0.0/0. port: 3000

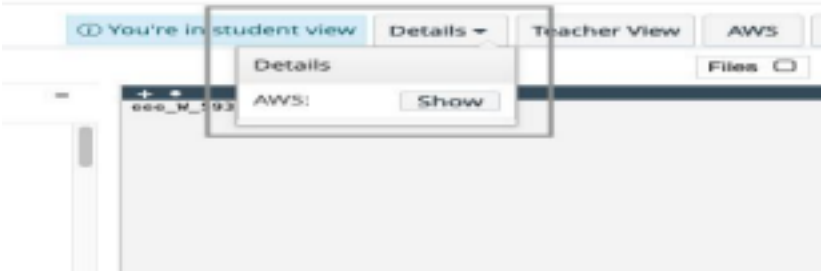


- Click Launch instance



4. Now Navigate to the Cloud Foundations – Sandbox AWS academy window

- Click on details then Show

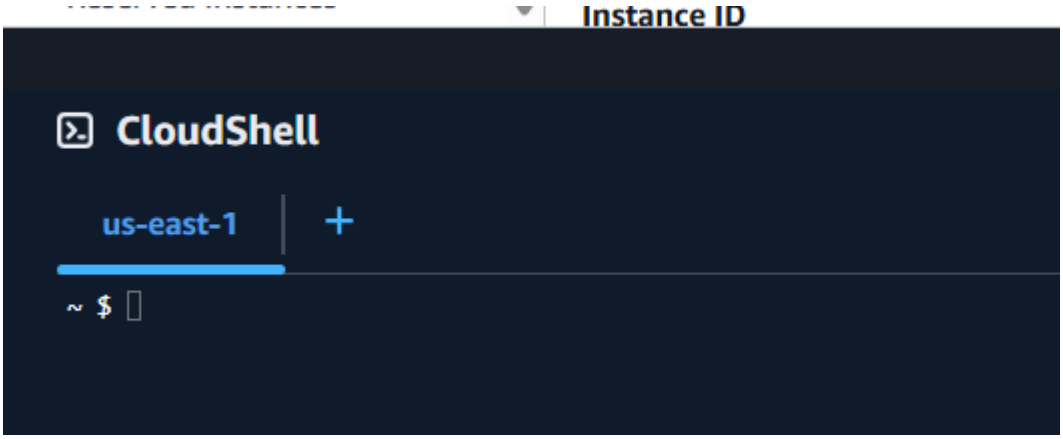


● Copy the whole key to access public EC2 through cloudshell.



```
-----BEGIN RSA PRIVATE KEY-----
MIIEowIBAAKCAQEAvk6zm2GavbyM9GZnqVFpv/EcvtKI3lnl8/FlzhrRkiEzypzg
jRH7SdoYiOgmkQx7TUDwA6+phnteM75rwaNcFRB3UMro0kKJI5ASB2hpQjBmeWPR
cJ2r9oEwS0HbVF5LRBq9/E9oAkVHrBOCjm13l5p/6Z7zKsoxb6zfpvalIAWAu6WN
bpmKZp6SoNAYAKQRwnxqGmSVKIkT4IWzFkAjyLWlQrQgvyxo6kwhX7xd2GWTe8To
tfet0wsCPGBU0EolWk1TgssYGO82gJsn7e/ryT6J0bbe8knx0UgkXJ80Q3gUoCoN
C6+ZdvqCxJNDmzLa8fukaxMX4QO301gJPBSSrwIDAQABAoIBAFOHPaP8giu3P3Li
I6uYdT7u+Dv1MmfNs3WPxaxh7wwhkJbkplakxtPSayRkKNFPVitBTld/2XuSWHWD
pc43AmqocIZAmVH4LDM0wAUKWfDYR5IOMtYoDltCtszBWiA8xvJcfnjo7XDZsayg
X5ReDbi38l67CmYEmxA2I3AoH6/el5N2iIpqmXgp6i2fldfFwSrcue3g62rRYOWZ
YYmFOgZMgdaHq5d38Z/9UiQzmU3PUF78c8mOUx8IEc/6Wre11lq4Ao3W0PJS9wKv
taLgNVIowqnkcMJ48x4ttHKVQSSe04f48rhugjzaZ26++oEECmVjicJHdIA5rN2a
At+oo2ECgYEA9bUWdgKn+9QoueYtBhQPn57O7ZccwqaAhzh9I+Via5JIaz/ttOna
1yJxlYPzJuBI93utcMSQ0dFjDBPcoCofYS4RrY1zygua2FrkM48yg/Rv79q+G3MS
i3sNDLOT3ulOjB3TgpxE6WIhO2n9iQy+mQsbcBPXqPTrwxFh0KZlKz8CgYEAxkeE
VOtHgJxPE5GxCdTEkS0RXFgJyenFKdWSHKlVp/K7FHr/JZh3ch26OK+nmJYe2bnd
nRKNXMXrae8gsLS41zIGAeA3/2Ut6L7auDh9hZIhNcZjJ2EMitwujlpmdb1FBehi
/oK8aGitoczaegrk030BaG6uVJdCkSrhqtnX7JECgYA4KYBM0SachU4Zz+cSsJsx
nRcnb5b/Vxeh0OOpxNDOTc2H1kD7TUjcS98ykcqOZE3JY9etIwEHNJkzNYcEhRxP
FjysMtz4ndZzBbOLpXuidcjwRkuV8qOWhjXhZVqzZRizJk4mWJnXzrpR3lQ5c1RN
nyBSafatg+4v1FVLX/1X+QKBgQC0VqGmduPutrw5aT5jWbB69kme+rfUfJmF5ijh
hqZQfbXn98KOmDCzixQvsJu3bXqbvrAwx4npQl5OWF8cORMRLv9Fv4BO+uCZmtTl
zwNjdDheZ5URYPR+a6qwcMpV9hKMQAOhxpjEQckCq4Zej//11vFr4ZcPew/hZuYf
dcks8QKBgHVAU8OpfDCKXPJeCc4eZs51vSOrxAbv3MRhZrVirytUPPnryShpM12+
/Z7KqMnHfTTdu7Z3Opx4xGFiWml70tstqszYGohlRy6fxsbdQsXRZ5pnRj9MJxyB
KpLHNJUuNp2wnPIXUVI8IOAbbux+0uHzDGPc8kJ4G/j0Oxs4lyDu
-----END RSA PRIVATE KEY-----
```

● Open Cloudshell terminal



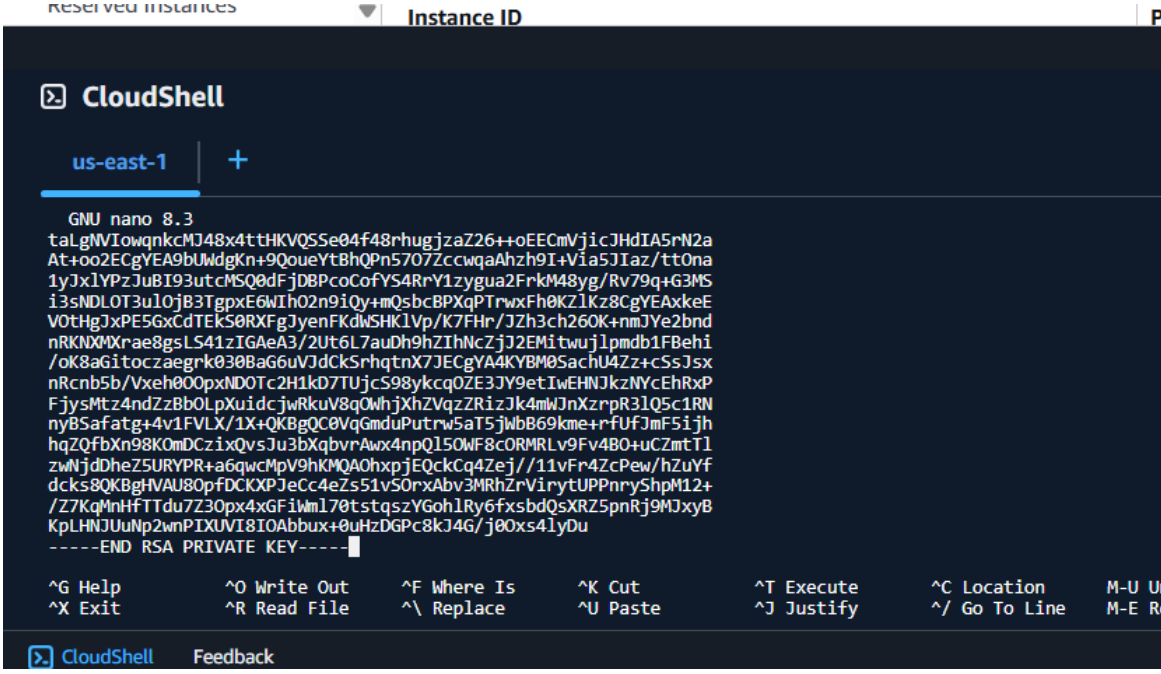
■ Create key file for access EC2 instance through SSH

◆ nano studentkey.pem



◆

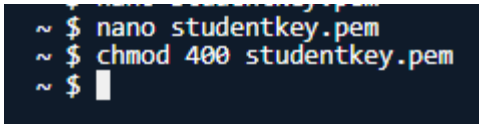
Just paste the key which you have copied from AWS academy sandbox environment window.



◆ Save the studentkey.pem file by Ctrl+Z and save it.

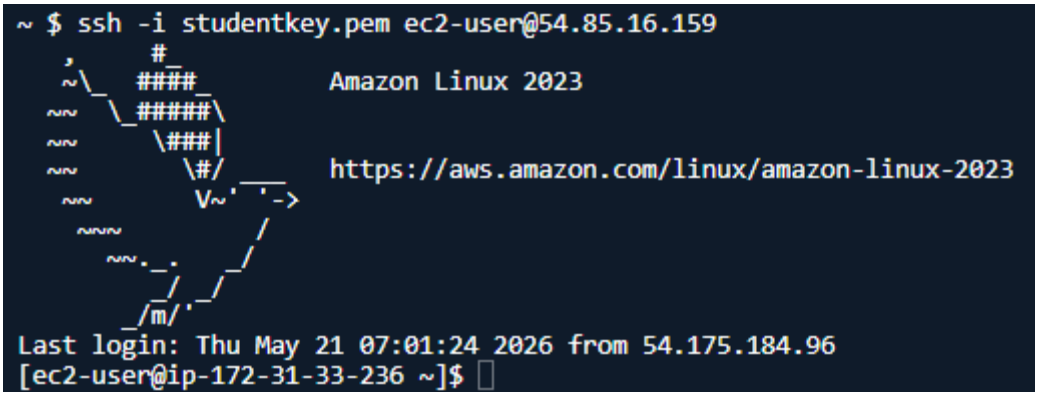
◆ Give the permission to key file

● chmod 400 studentkey.pem



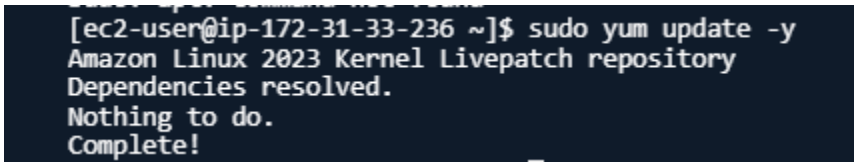
◆ login to ec2 instance through SSH

● ssh -i studentkey.pem ubuntu@PUBLIC-IP



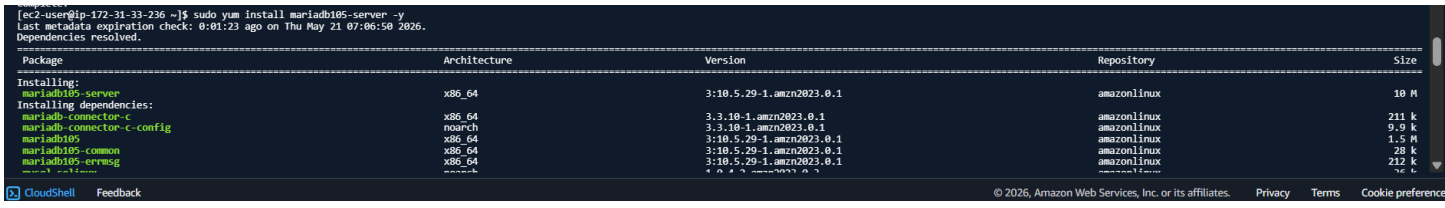
5. Update ubuntu

● sudo apt update -y



6. Install MySQL server

● sudo apt install mysql-server -y



◆ Check version

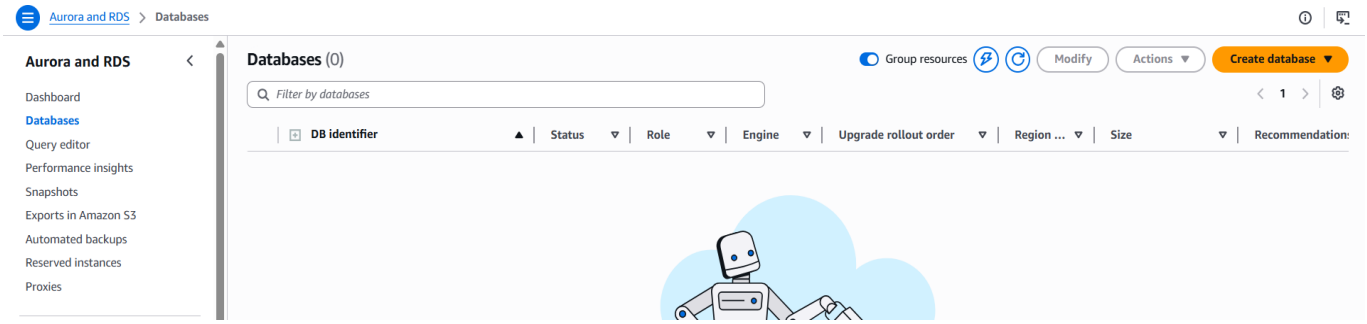
● mysql --version

```
Complete!
[ec2-user@ip-172-31-33-236 ~]$ mysql --version
mysql  Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using  EditLine wrapper
[ec2-user@ip-172-31-33-236 ~]$
```

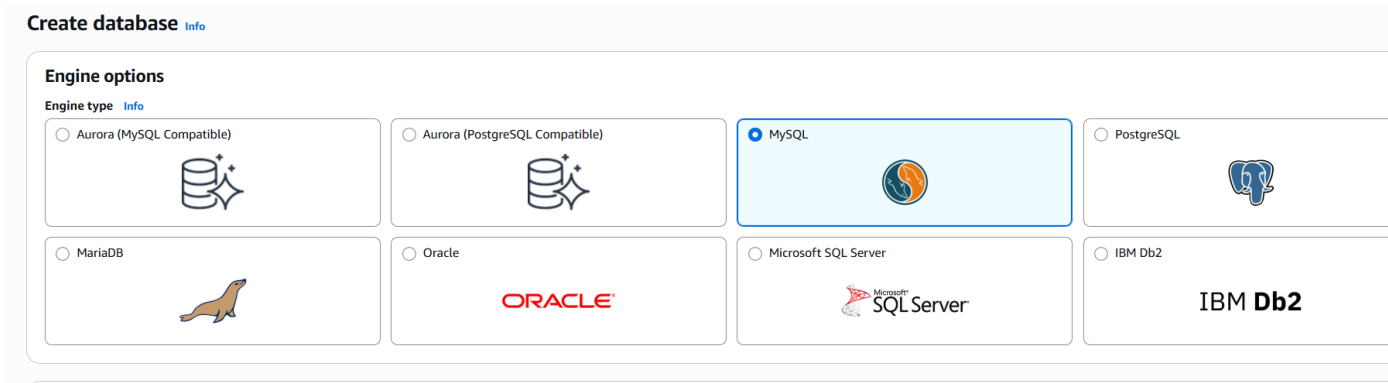
EIU

Task 2: Create MySQL database accessible from anywhere.

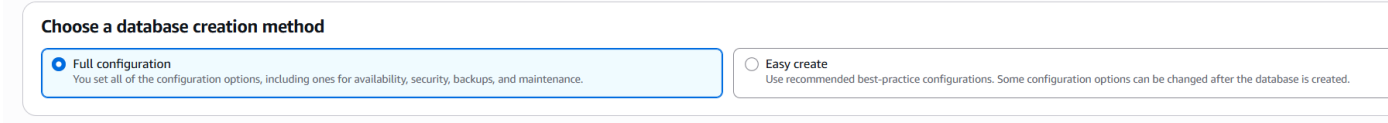
☑ Open AWS Console → Aurora and RDS → Databases → Create Database



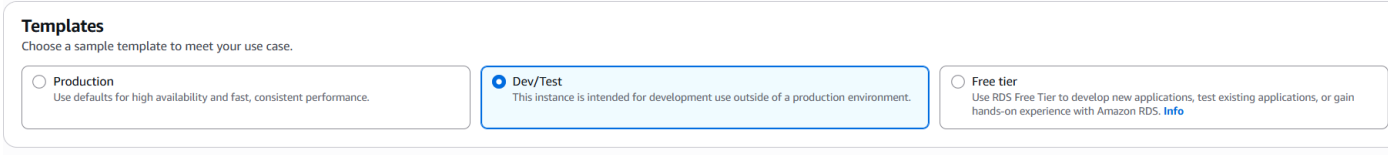
☑ Select Engine type as **MySQL**



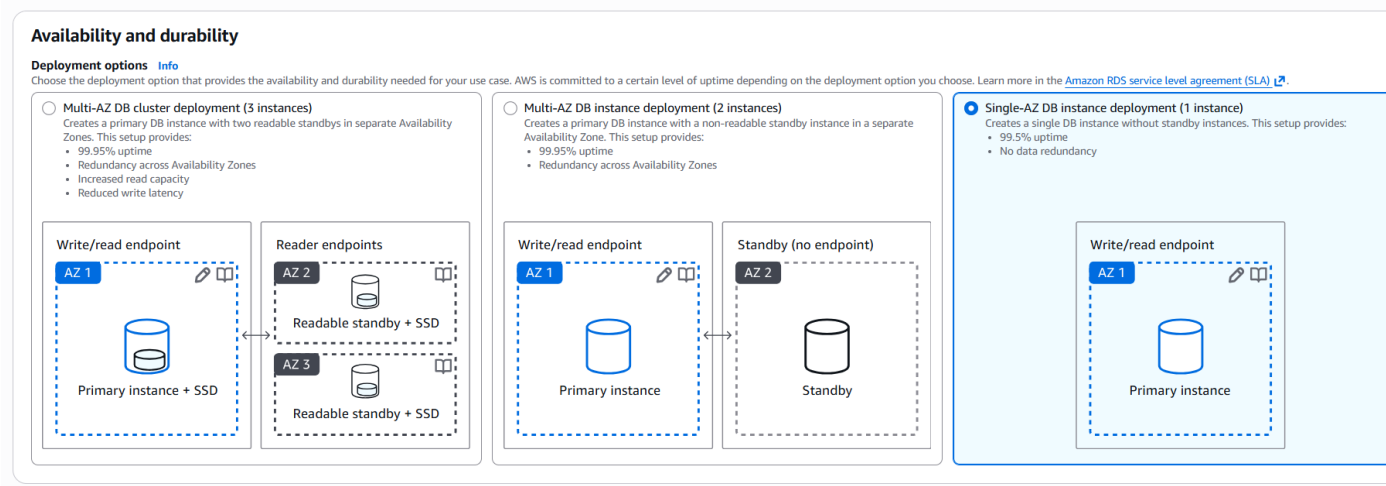
☑ Choose a database creation method select → Full configuration



☑ Templates → choose Dev/test

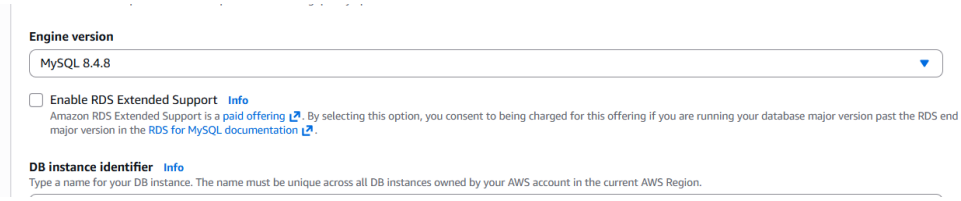


☑ Availability and durability → Deployment options → Single-AZ DB instance deployment (1 instance)

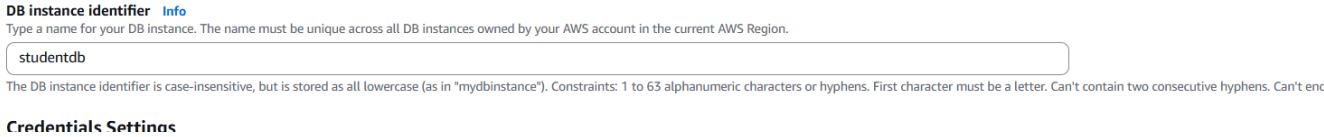


Settings

- Default MySQL community, Version leave default



- In DB instance identifier set name as **student db**



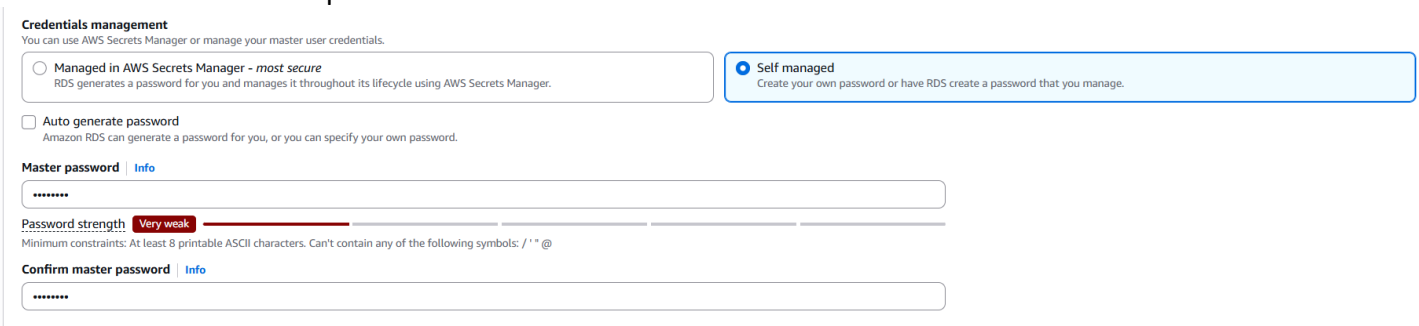
Credentials Settings

- Credentials Settings
  - master username set as **admin**

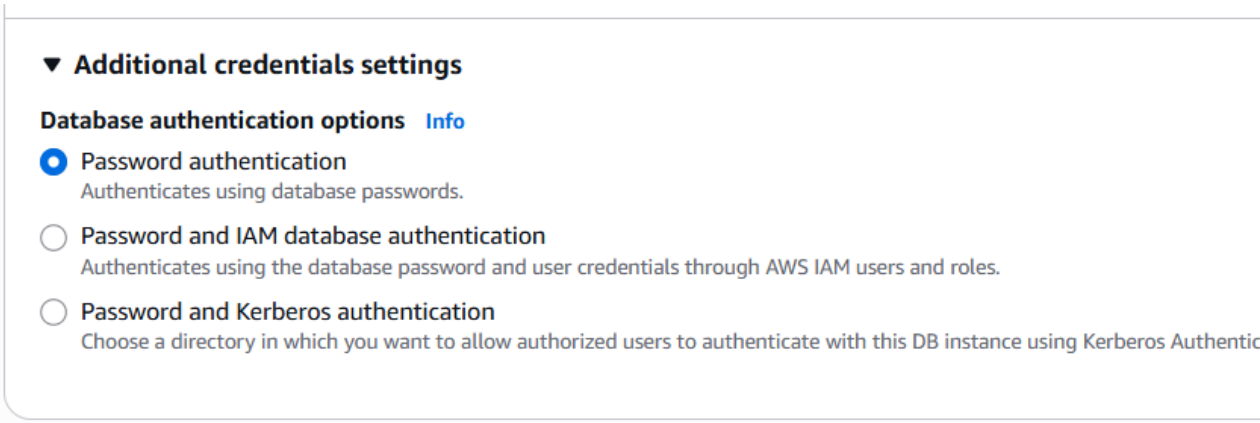


Credentials management

  - Credentials management: select **self managed**
    - Make password: admin123

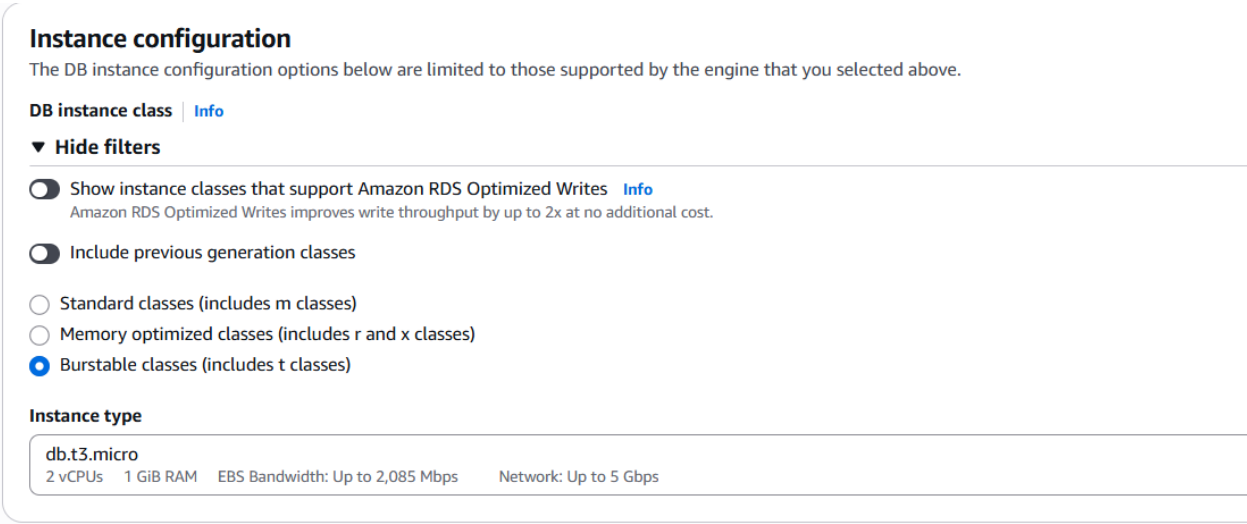


  - Additional credential settings: select **password authentication**



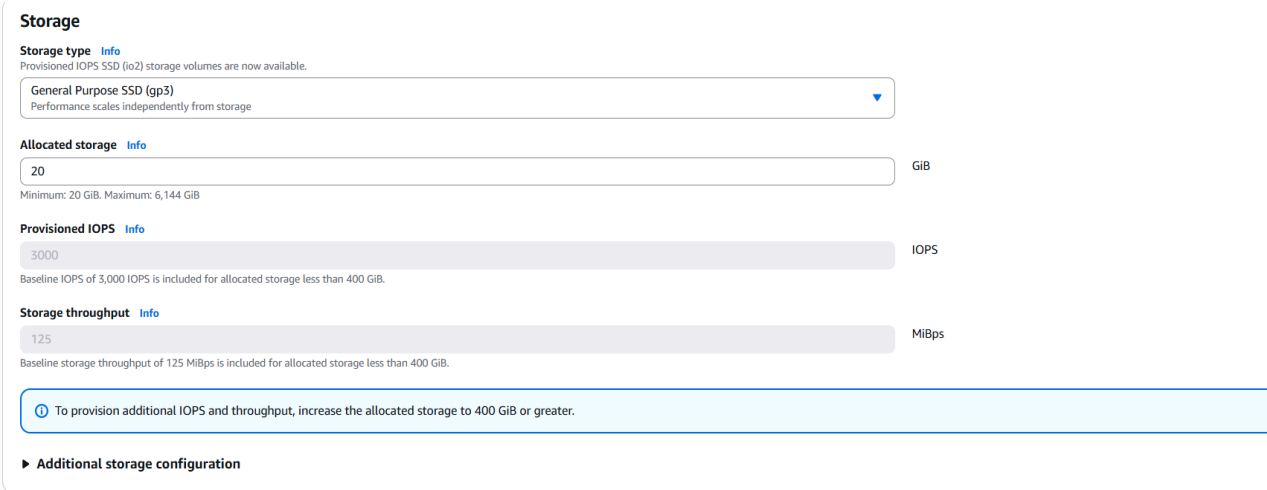
☑Instance configuration

- Select **Burstable classes (includes t classes)**
- Instance type: Select db.t3.micro



☑Storage configure

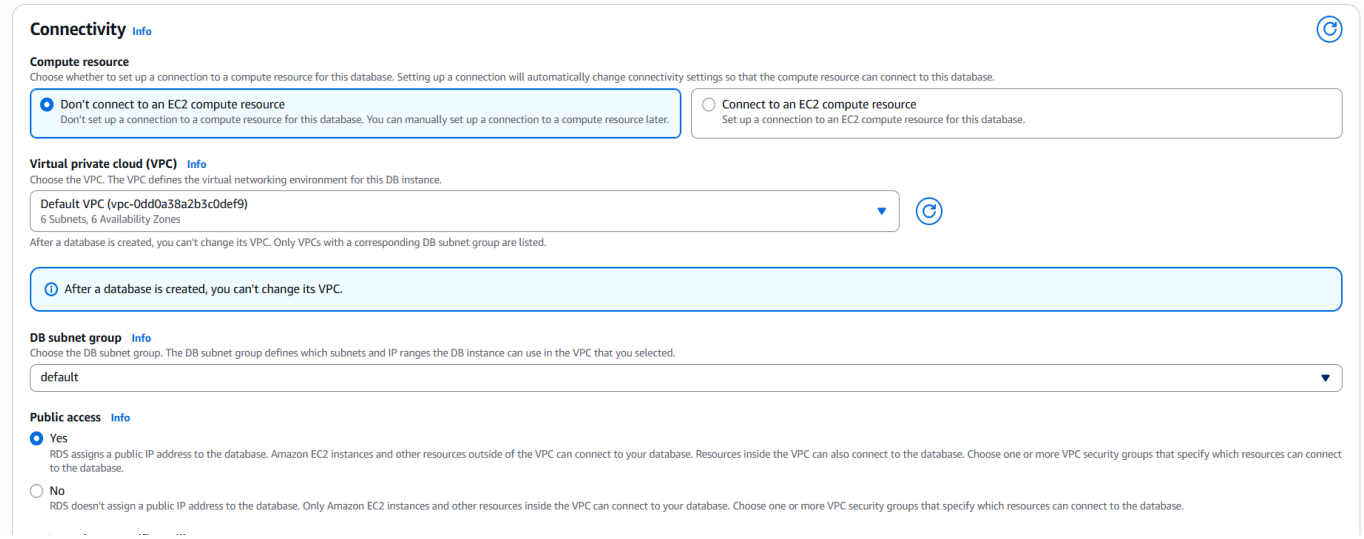
- Storage type: General Purpose (SSD) gp3
- Allocated storage: 20 GiB
- Other setting leave as default.



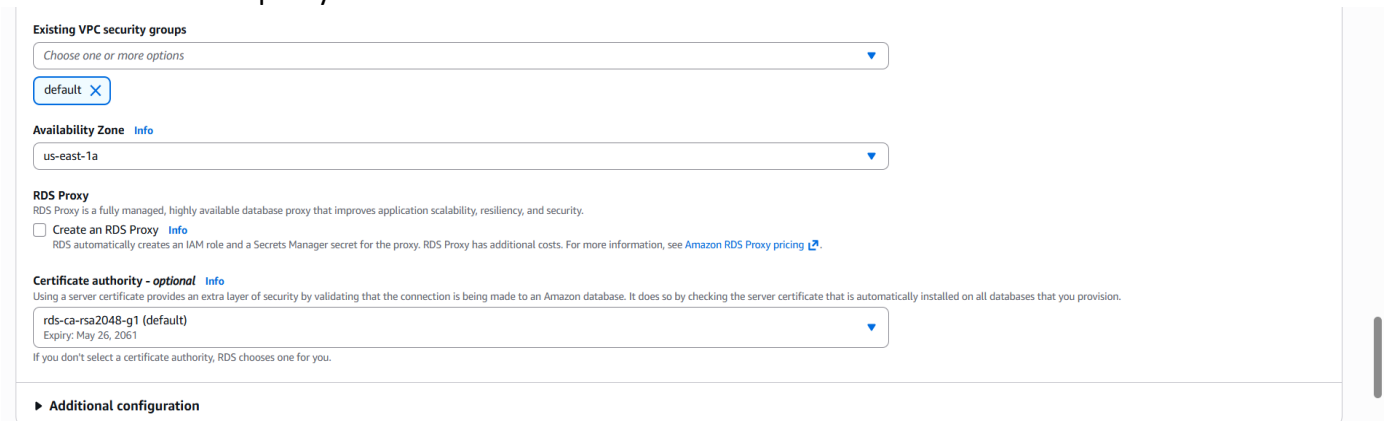
☑Connectivity:

- ◆ Compute resources

- Select Don’t connect to an EC2 compute resource
- VPC: default
- subnet group leave as default
- Public access: Yes

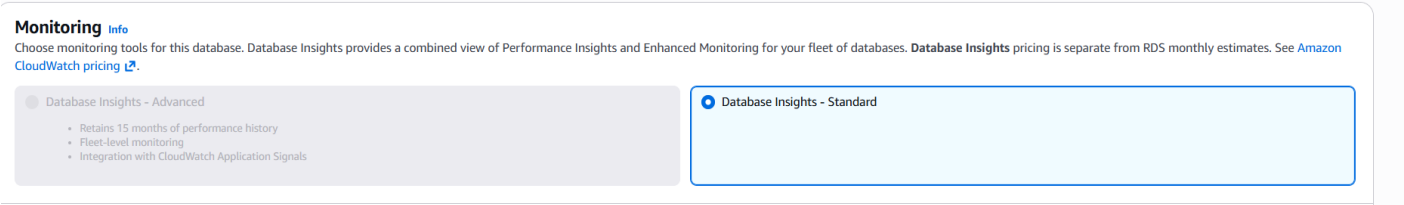


- Leave other settings as defaults.
- Availability Zone
  - select US-east-1a
- RDS proxy leave as default

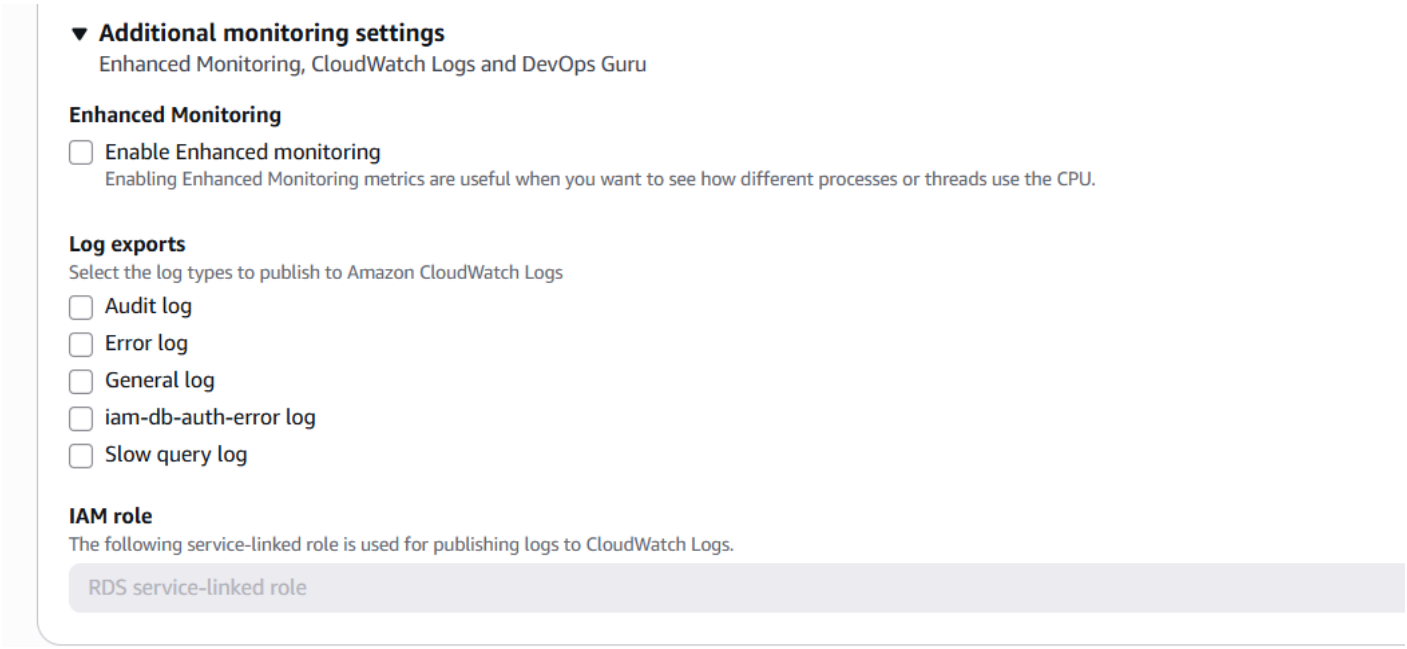


Monitoring

- select: Database Insights - Standard

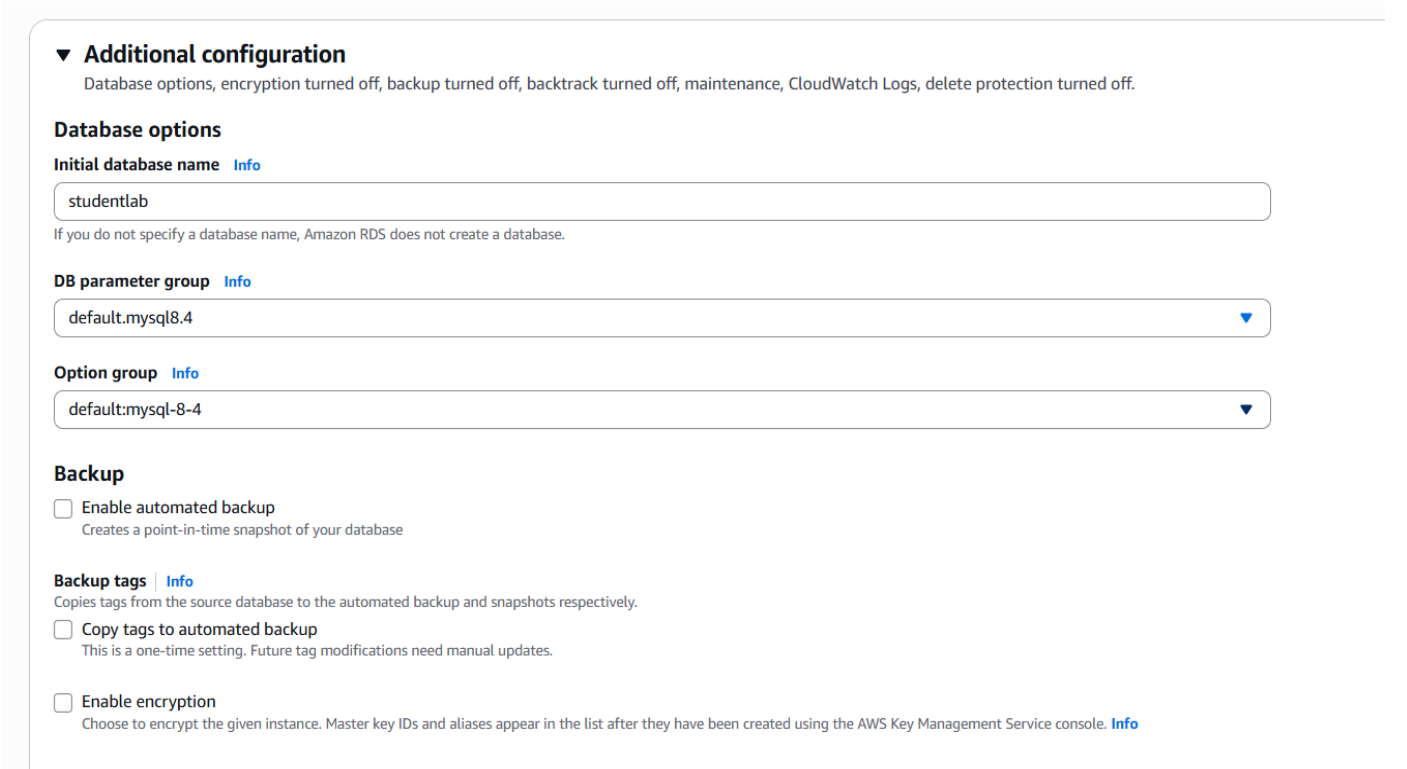


- Under Monitoring expand Additional configuration.
  - Uncheck Enable Enhanced monitoring.

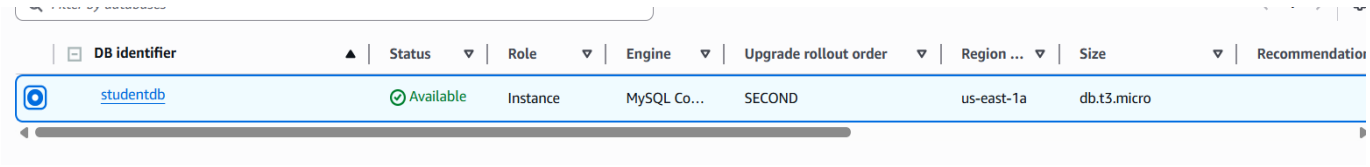


◆ Under Additional configuration, configure

- Initial database name: studentlab
- DB parameter group leave as default
- Uncheck Enable automatic backups.
- Uncheck Enable encryption

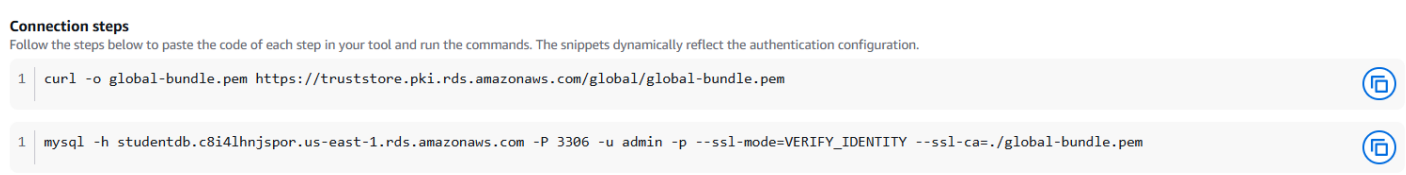


Click Create and wait until the status is Available.



Save for later:

- Endpoint
- Port: 3306
- DB name: studentlab
- User: admin
- Password: admin123




```
mysql -h studentdb.c8i4lhnjspor.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem
```

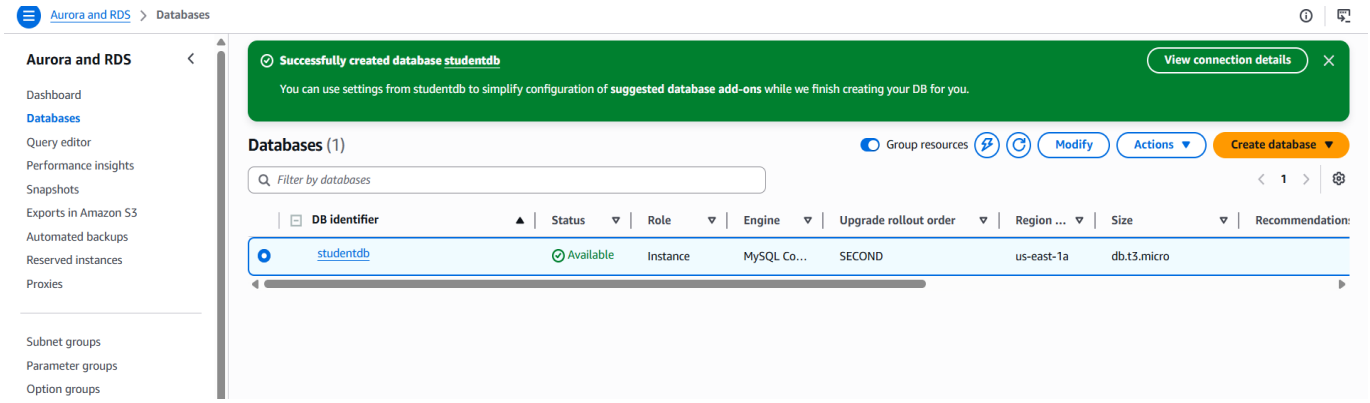
You'll create the schema later via RDS Query Editor v2.

Task 3: Connect MySQL database through EC2 instances.

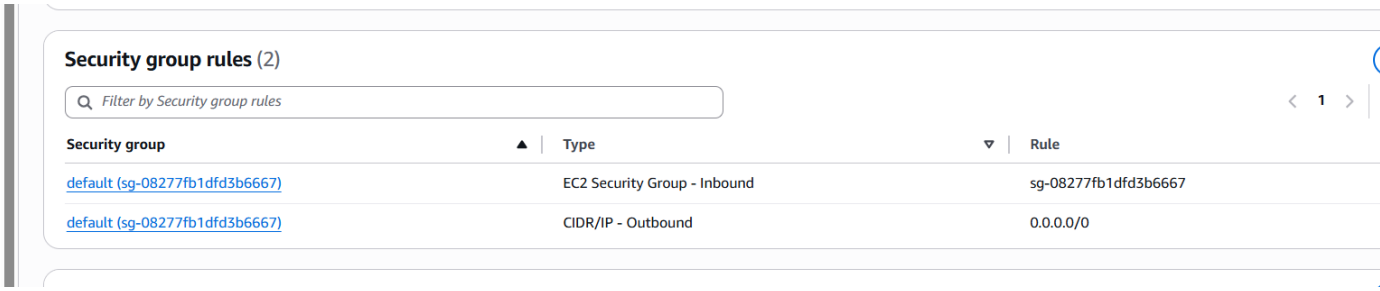
Go back to cloudshell.

Access to EC2 Ubuntu instance through SSH Before connecting to database through EC2 set the Inbound security rule of Database.

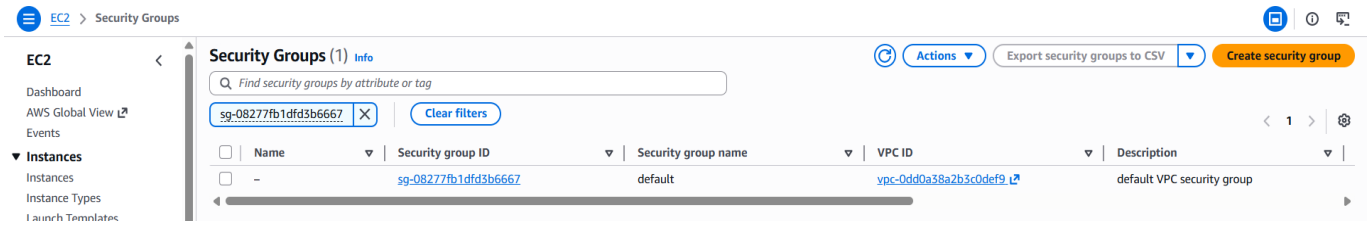
- Go to database.
  - Select created studentdb



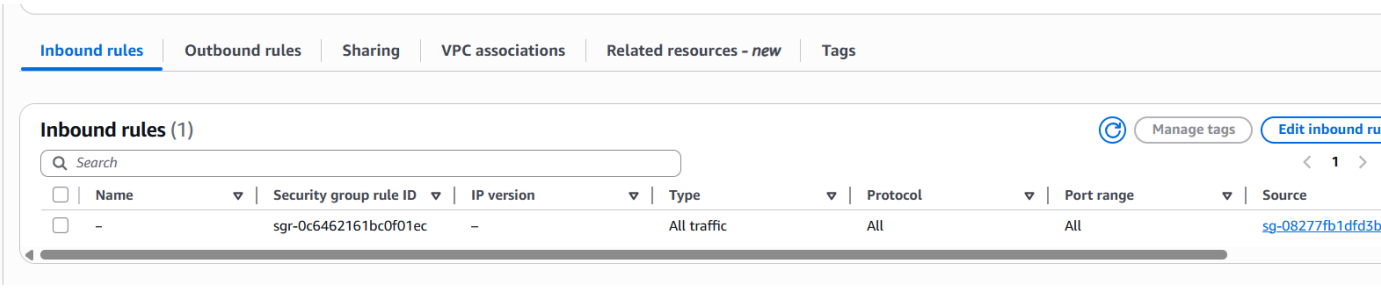
  - Scroll down and in the Security group rules section



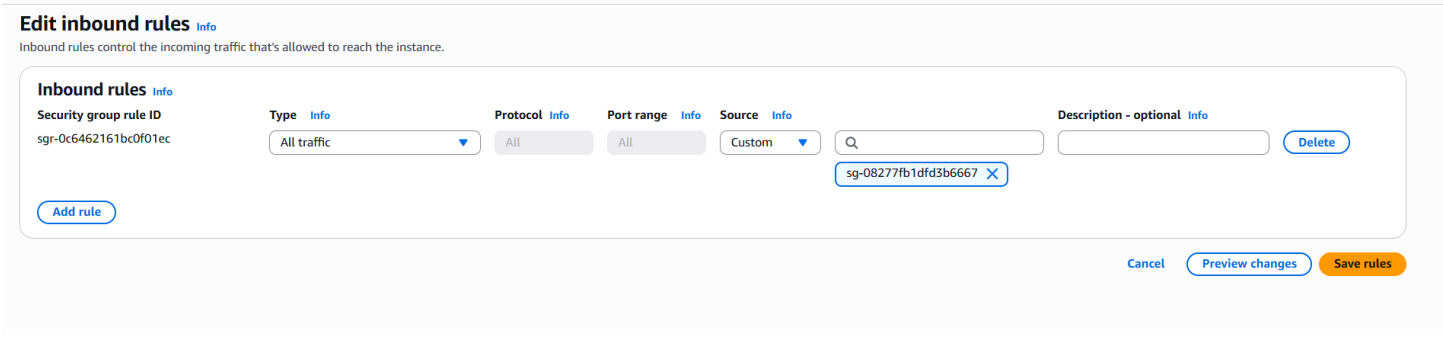
■ click on security of type EC2 Security Group - Inbound



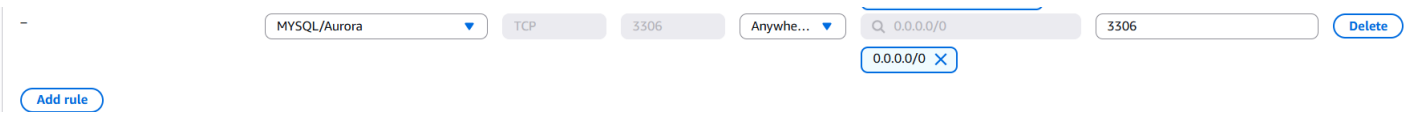
◆ Scroll down and click on Tab Inbound security rules



● Then click on Edit inbound rules



● Click Add security group rule. Type MYSQL/Aurora, Source Custom0.0.0.0/0. port: 3306



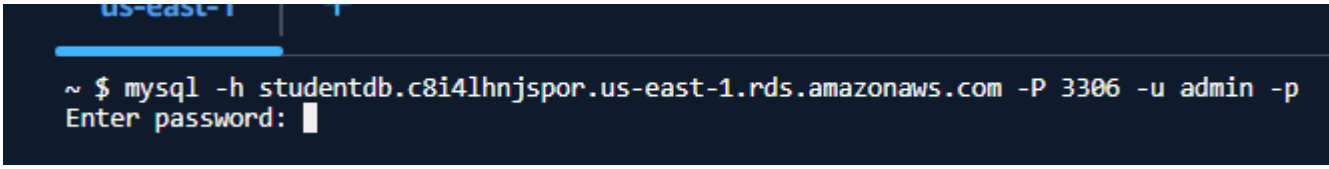
● Save the security rules.



- Copy your database endpoint

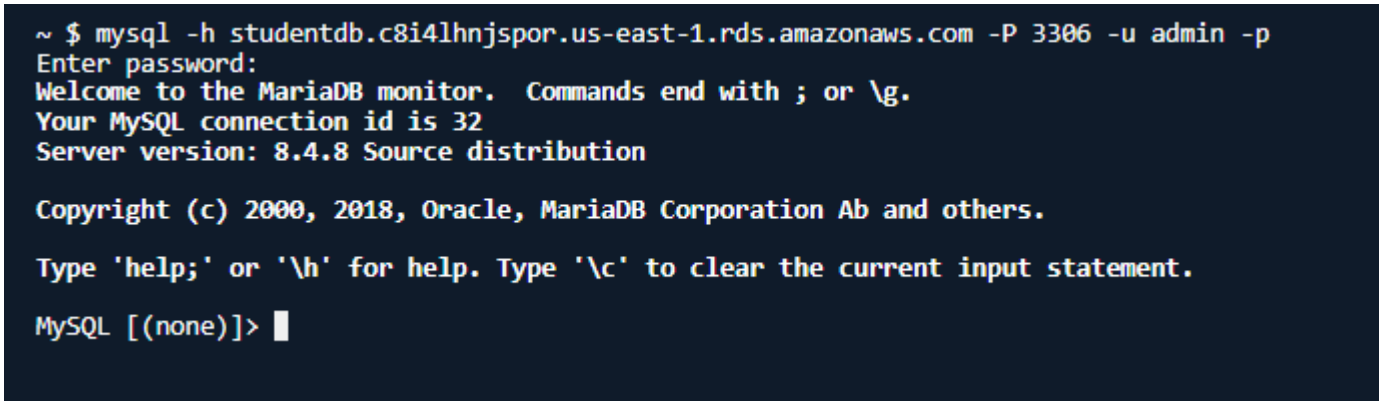
studentdb.c8i4lhnjspor.us-east-1.rds.amazonaws.com

- Now connect to database using command
  - mysql -h paste your RDS endpoint address -P 3306 -u admin -p



  - Enter your database password

  - Now you connected to MySQL database.



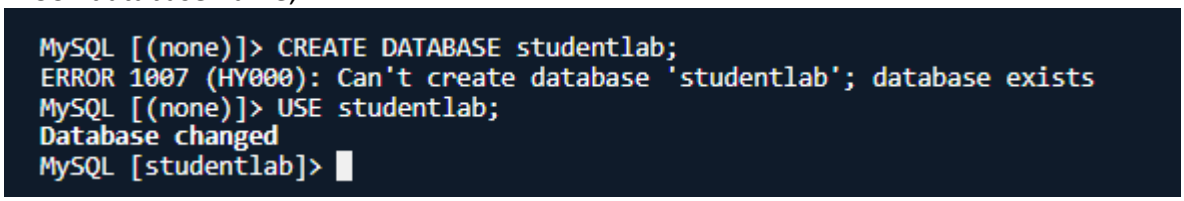
Task 4: Create Database, Table and insert record in the table.

Create Database inside MySQL using command

- CREATE DATABASE database name;

Use database using command

- USE database name;



Create Table using command

CREATE TABLE students (

id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(50),

course VARCHAR(50)

);

```
Database changed
MySQL [studentlab]> CREATE TABLE students (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     name VARCHAR(50),
    ->     course VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.064 sec)

MySQL [studentlab]>
```

Insert record in to the table

INSERT INTO students(name, course)

VALUES

('John', 'AWS'),

('Alice', 'Cloud Computing'),

('Bob', 'DevOps');

```
MySQL [studentlab]> INSERT INTO students(name, course)
    -> VALUES
    -> ('John', 'AWS'),
    -> ('Alice', 'Cloud Computing'),
    -> ('Bob', 'DevOps');
Query OK, 3 rows affected (0.007 sec)
Records: 3  Duplicates: 0  Warnings: 0

MySQL [studentlab]>
```

Verify the inserted data

SELECT * FROM students; (if table name is students)

```
MySQL [studentlab]> SELECT * FROM students;
+----+-------+-----------------+
| id | name  | course          |
+----+-------+-----------------+
|  1 | John  | AWS             |
|  2 | Alice | Cloud Computing |
|  3 | Bob   | DevOps          |
+----+-------+-----------------+
3 rows in set (0.002 sec)

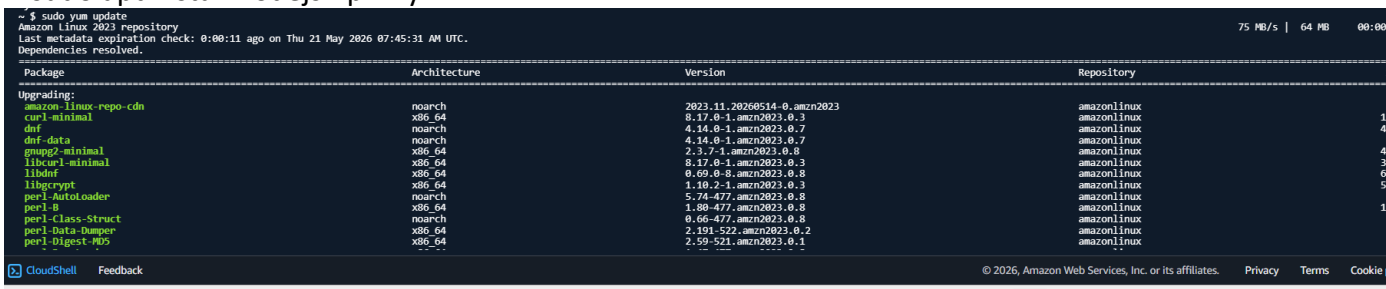
MySQL [studentlab]>
```

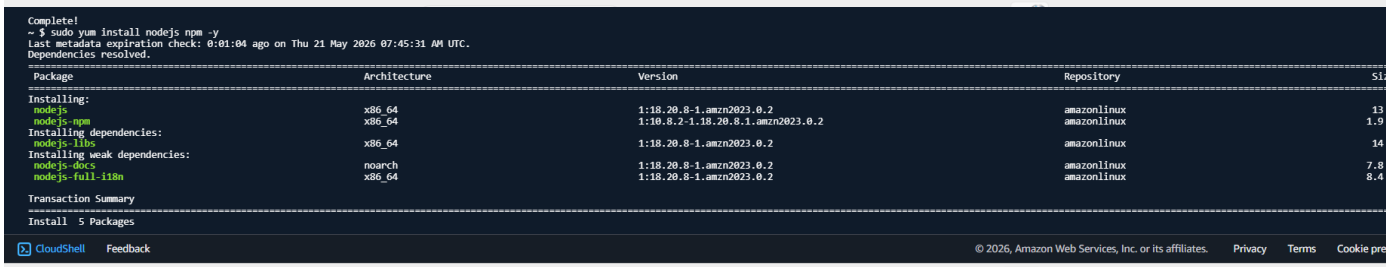
After creating table and inserting data exit from SQL using EXIT;

```
MySQL [studentlab]> EXIT;
Bye
~ $
```

Task 5: Display the student record on the web page through EC2 web server 1. Install NodeJS on EC2

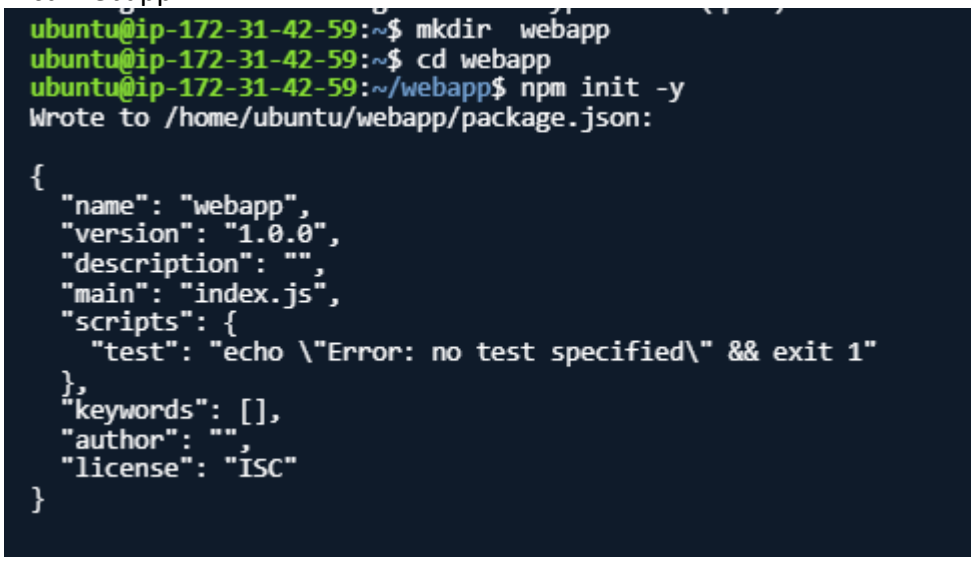
- sudo apt update
- sudo apt install nodejs npm -y





2. Create web project folder using command

- mkdir webapp
- cd webapp



3. Initialize project

- npm init -y

```
ubuntu@ip-172-31-42-59:~$ mkdir  webapp
ubuntu@ip-172-31-42-59:~$ cd webapp
ubuntu@ip-172-31-42-59:~/webapp$ npm init -y
Wrote to /home/ubuntu/webapp/package.json:

{
  "name": "webapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

4. Install npm packages and create app.js file
   - npm install express mysql2

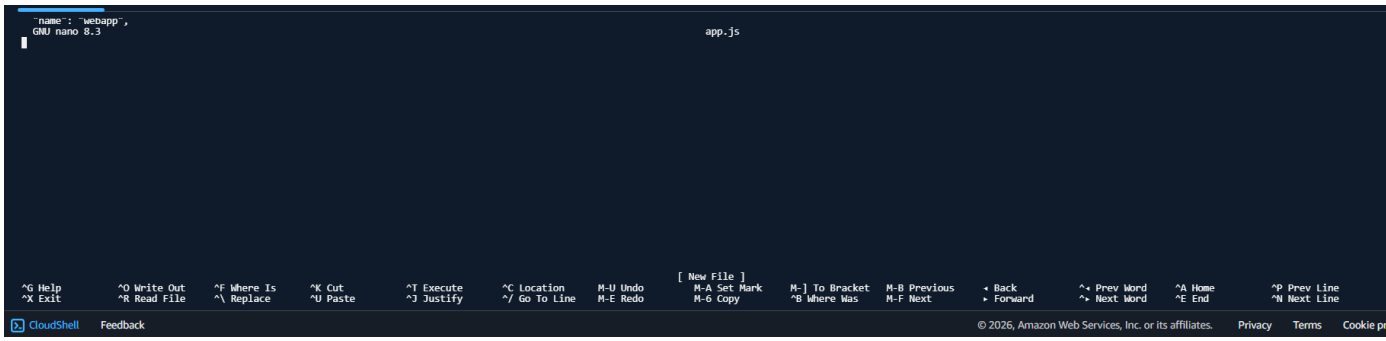
```
ubuntu@ip-172-31-42-59:~/webapp$ npm install express mysql2

added 77 packages, and audited 78 packages in 3s

26 packages are looking for funding
  run `npm fund` for details
```

   - create app.js using

nano app.js



   - paste the following code in the app.js file.

```
const express = require('express');
const mysql = require('mysql2');

const app = express();
const db = mysql.createConnection({
   host: 'paste your database endpoint url',
   user: 'admin',
   password: 'database password',
   database: 'database name'
});
```

```
db.connect((err) => {
    if(err){
        console.log(err);
    } else {
        console.log('RDS Connected');
    }
});

app.get('/', (req, res) => {

    db.query('SELECT * FROM students', (err, results) => {
                                                        res.send(err);
                                                        } else {

                                                        let html = ` <html> <head>
        if(err){




                <title>Student Records</title> </head>

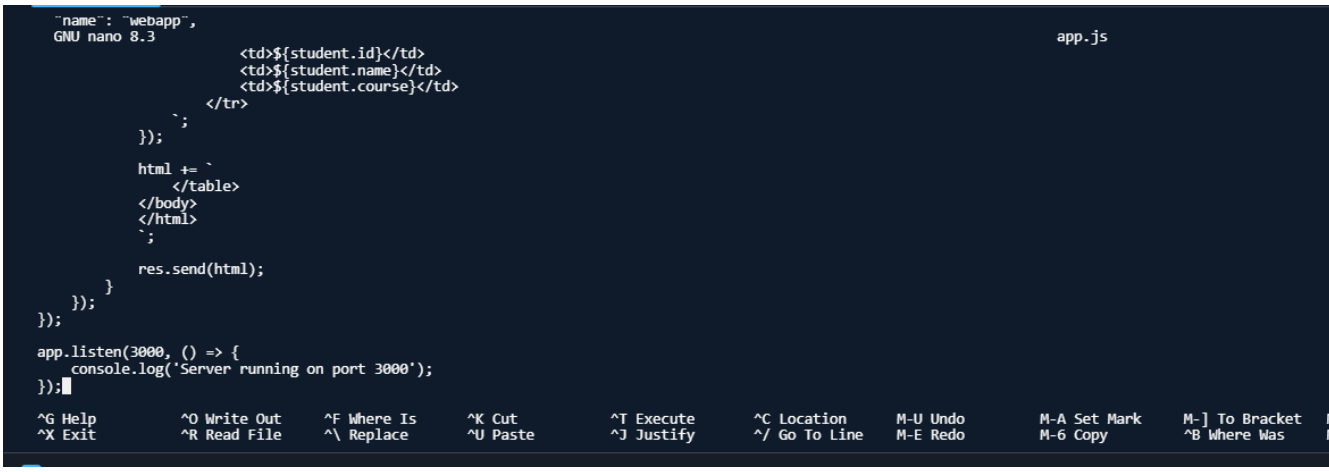
            <body>

                <h1>Student Records</h1> <table border="1" cellpadding="10">

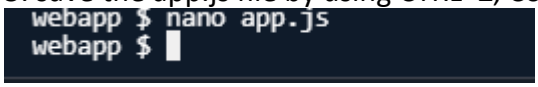
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Course</th>
                    </tr>
            `;
            results.forEach(student => {
                html += `
                    <tr>
                        <td>${student.id}</td>
                        <td>${student.name}</td> <td>${student.course}</td>
                    </tr>
                `;
            });
```

```
            html += `
               </table>
            </body>
            </html>
            `;
            res.send(html);
        }
    });
});
app.listen(3000, () => {

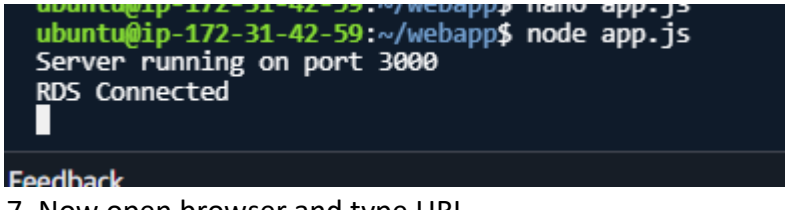
    console.log('Server running on port 3000');
});
```



5. save the app.js file by using CTRL+Z, Confirm Y (Yes) and press enter

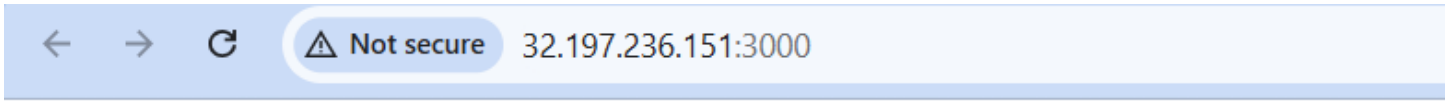


6. Run your app.js
   - node app.js
   - It will display: RDS Connected
     Server running on port 3000



7. Now open browser and type URL
   http://EC2-PUBLIC-IP:3000
   http://32.197.236.151:3000/

8. You should now see:
   - Student Records webpage
   - Data loaded from RDS database



Student Records

ID	Name	Course
1	John	AWS
2	Alice	Cloud Computing
3	Bob	DevOps

Task 6: Deploying a Redis Cache-Based Web Application on AWSEC2

1. If EC2 instance not launched then launched EC2 instance using the steps of Task 1 fromsteps1 to step 5.
   a) Then install Redis using command: **sudo apt install redis-server -y**

```
ubuntu@ip-172-31-42-59:~$ sudo apt install redis-server -y
Installing:
  redis-server

Installing dependencies:
  liblzf1  redis-tools

Suggested packages:
  ruby-redis

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 30
  Download size: 1431 kB
  Space needed: 7481 kB / 3808 MB available

Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 liblzf1 amd64 3.6-4build1 [7502 B]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 redis-tools amd64 5:8.0.5-1 [1370 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 redis-server amd64 5:8.0.5-1 [53.2 kB]
Fetched 1431 kB in 0s (33.8 MB/s)
Selecting previously unselected package liblzf1:amd64.
(Reading database ... 126412 files and directories currently installed.)
Preparing to unpack .../liblzf1_3.6-4build1_amd64v3.deb ...
Unpacking liblzf1:amd64 (3.6-4build1) ...
Selecting previously unselected package redis-tools.
Preparing to unpack .../redis-tools_5%3a8.0.5-1_amd64v3.deb ...
Unpacking redis-tools (5:8.0.5-1) ...
Selecting previously unselected package redis-server.
Preparing to unpack .../redis-server_5%3a8.0.5-1_amd64v3.deb ...
Unpacking redis-server (5:8.0.5-1) ...
```

Feedback

   b) Start Redis using command: **sudo systemctl start redis-server**
   c) Enable auto-start: **sudo systemctl enable redis-server**
   d) Check status: **sudo systemctl status redis-server**

```
ubuntu@ip-172-31-42-59:~$  sudo systemctl start redis-server
ubuntu@ip-172-31-42-59:~$  sudo systemctl enable redis-server
Synchronizing state of redis-server.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable redis-server
ubuntu@ip-172-31-42-59:~$  sudo systemctl status redis-server
● redis-server.service - Advanced key-value store
     Loaded: loaded (/usr/lib/systemd/system/redis-server.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-05-21 08:35:42 UTC; 1min 17s ago
 Invocation: 644b567a65944326bd942424b936d094
       Docs: http://redis.io/documentation,
             man:redis-server(1)
   Main PID: 7672 (redis-server)
     Status: "Ready to accept connections"
      Tasks: 6 (limit: 657)
     Memory: 4.8M (peak: 4.8M)
        CPU: 313ms
     CGroup: /system.slice/redis-server.service
             └─7672 "/usr/bin/redis-server 127.0.0.1:6379"

May 21 08:35:42 ip-172-31-42-59 systemd[1]: Starting redis-server.service - Advanced key-value store...
May 21 08:35:42 ip-172-31-42-59 systemd[1]: Started redis-server.service - Advanced key-value store.
```

2. Test Redis.

a) using command: **redis-cli**

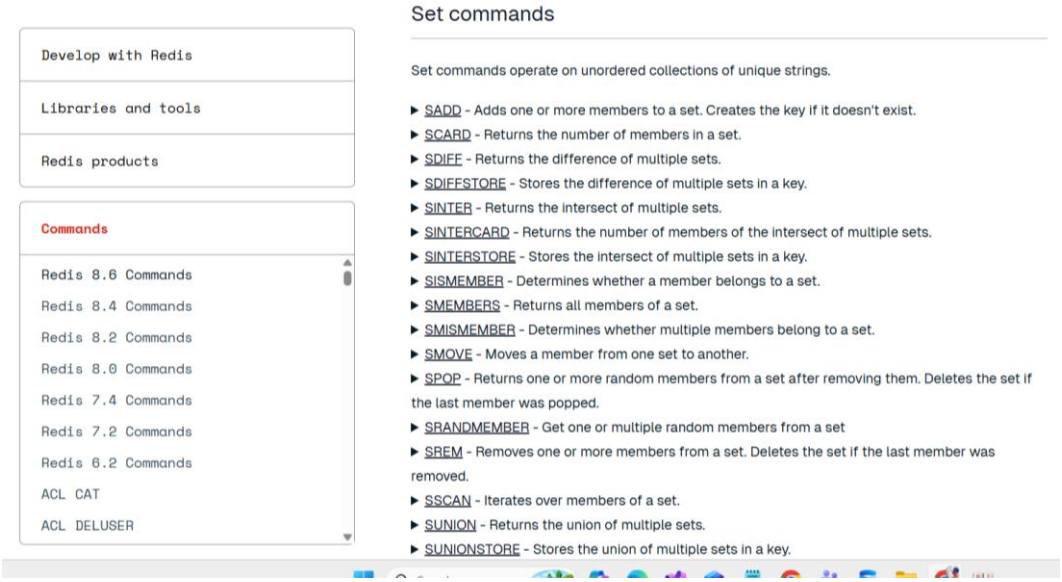
```
ubuntu@ip-172-31-42-59:~$ redis-cli
127.0.0.1:6379>
```

b) Store data in to the redis: **SET course AWS** (This command is used for storing stringtype data).

```
127.0.0.1:6379> SET course AWS
OK
127.0.0.1:6379>
```

c) For more way of storing data in Redis follow the below link document

- https://redis.io/docs/latest/commands/redis-8-6-commands/#set-commands



d) Now retrieve the data using command: GET course (course is the key)

```
127.0.0.1:6379> GET course
"AWS"
127.0.0.1:6379>
```

e) Exit from redis cli using **exit** command

```
127.0.0.1:6379> exit
ubuntu@ip-172-31-42-59:~$
```

3. Install node JS if ot install in EC2 instance server.

a) sudo apt install nodejs npm -y

```
ubuntu@ip-172-31-42-59:~$ sudo apt install nodejs npm -y
nodejs is already the newest version (22.22.1+dfsg+~cs22.19.15-1ubuntu1).
npm is already the newest version (9.2.0~ds3-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 30
ubuntu@ip-172-31-42-59:~$
```

b) Create web application folder using command:

i. **mkdir redisapp**

ii. then navigate to folder using command: **cd redisapp**

```
ubuntu@ip-172-31-42-59:~$ mkdir redisapp
ubuntu@ip-172-31-42-59:~$ cd redisapp
ubuntu@ip-172-31-42-59:~/redisapp$
```

c) Initialize project: **npm init -y**

```
ubuntu@ip-172-31-42-59:~/redisapp$ npm init -y
Wrote to /home/ubuntu/redisapp/package.json:

{
  "name": "redisapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

d) Install packages: **npm install express redis**

```
ubuntu@ip-172-31-42-59:~/redisapp$ npm install express redis

added 73 packages, and audited 74 packages in 4s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
ubuntu@ip-172-31-42-59:~/redisapp$
```

e) Create app.js file using command: **nano app.js**

f) Copy and paste the below code in to the app.js file.

```
const express = require('express');
const redis = require('redis');

const app = express();
```

```
const client = redis.createClient();

(async () => {

    await client.connect();

    console.log('Redis Connected');

    await client.set('message', 'Welcome to Redis Cache on AWS EC2'); })();

app.get('/', async (req, res) => {

    const value = await client.get('message');

    res.send(`
        <html>
        <head>
            <title>Redis Demo</title>
        </head>

        <body>

            <h1>Redis Cache Demo</h1>

            <p>${value}</p>

        </body>
        </html>
    `);
});

app.listen(3000, () => {

    console.log('Server running on port 3000');

});
```



g) Save app.js file and run the application using command: **node app.js**



h) Now open browser and type URL

i. http://EC2-PUBLIC-IP:3000
http://32.197.236.151:3000/

i) Just see the redis data results in the browser.



Task 7: Reflections Questions

- After implementing the web applications using Amazon RDS, and Redis on AmazonEC2, compare the differences between relational databases (RDS MySQL), and in-memory cached databases (Redis) in terms of data structure, storage method, performance, scalability, and real-world use cases.

Answer

After deploying the web application with Amazon RDS (MySQL) for persistent storage and Redis on Amazon EC2 for in-memory caching, the differences between the two technologies become clear. While both serve as data layers, they are designed for fundamentally different purposes.

1. Data Structure

RDS MySQL is a relational database that organizes information into tables consisting of rows and columns. Each table follows a strict, predefined schema, and relationships between tables are enforced through primary and foreign keys. This makes it well-suited for structured data with clear relationships, such as students, courses, and enrollments.

Redis, in contrast, is a key-value store that supports multiple flexible data structures including strings, lists, sets, sorted sets, hashes, bitmaps, and streams. There is no fixed schema, which allows developers to model data in whichever form best fits the use case.

2. Storage Method

MySQL stores data persistently on disk. In Amazon RDS, this corresponds to EBS-backed storage, which ensures that data survives reboots, failures, and instance changes. Backups, snapshots, and replication are managed automatically by AWS.

Redis stores data primarily in RAM, which is the source of its remarkable speed. Persistence to disk is optional and can be enabled through RDB snapshots or Append-Only File (AOF) logs, but Redis is fundamentally optimized for ephemeral, fast-access workloads.

3. Performance

Because MySQL reads from and writes to disk and parses SQL queries, its operations are typically measured in milliseconds. This is fast enough for most transactional workloads but cannot match the speed of an in-memory system.

Redis operates entirely in memory and serves requests with sub-millisecond latency, capable of handling hundreds of thousands to millions of operations per second. This makes Redis ideal as a cache layer in front of a slower database such as MySQL.

4. Scalability

RDS MySQL scales primarily through vertical scaling — upgrading to a larger instance type with more CPU, memory, or storage. Horizontal scaling is possible using read replicas or sharding, but these approaches introduce significant complexity, especially when maintaining ACID guarantees.

Redis is designed for horizontal scalability. Through Redis Cluster, data can be partitioned across many nodes, and replication is simple to configure. This makes Redis particularly suitable for distributed systems with very high throughput requirements.

5. Real-World Use Cases

RDS MySQL is the right choice when:

- Data must be stored permanently and reliably.
- Relationships, joins, and transactions are essential.
- Strong consistency and ACID compliance are required.

Typical examples include online banking systems, university management systems, and e-commerce order processing — exactly the kind of workload represented by the student database deployed in this lab.

Redis is the right choice when:

- Response time must be extremely low.
- Frequently accessed data should be cached to reduce load on the primary database.
- Real-time features such as leaderboards, live chats, or session tracking are needed.